DeepStream environment setup

DeepStream environment setup

1. Checks before the installation
2. Download the DeepStream 9.1 installation package
3. Install the TensorRT 10.x dependencies
4. Install DeepStream 9.1
5. Verify the installation
6. Run the basic video detection case
7. USB and CSI camera detection
    - 7.1. USB camera
    - 7.2. CSI camera
8. Clean up the installation package
9. Common problems
    - Problem 1: aria2c cannot be found
    - Problem 2: dependency error
    - Problem 3: TensorRT version conflict

> This tutorial applies to the Jetson Orin NX with JetPack 7.2.1 / Ubuntu 24.04 / L4T R39.2 / CUDA 13.2. The factory system already has the relevant environment set up, so this tutorial can be skipped.

1. Checks before the installation

First confirm that the system is already on the JetPack 7.2.1 baseline and check the TensorRT dependencies. DeepStream 9.1 requires CUDA 13.2 and TensorRT 10.16; if 10.16.2 is not present, the 10.16.2 version of TensorRT has to be installed.

```
cat /etc/os-release
dpkg-query -W -f='${Package} ${Version}\n' \
  nvidia-l4t-core libnvinfer10 libnvonnxparsers10 libnvinfer-plugin10
df -h /
```

The version of the target machine actually verified in this course:

```
Ubuntu 24.04.4 LTS
L4T R39.2.1
CUDA 13.2
TensorRT 10.16.2
```

Before the installation, reserve at least 5 GB of root partition space; more space is needed later when downloading the advanced models and building the TensorRT engines.

2. Download the DeepStream 9.1 installation package

The DeepStream 9.1 Jetson arm64 installation package is published in the official NVIDIA GitHub Release. Using `aria2c` for the multi-connection download is recommended.

```
cd /tmp
aria2c -x 16 -s 16 -k 1M \
  https://github.com/NVIDIA/DeepStream/releases/download/v9.1.0/deepstream-9.1_9.1.0-1_arm64.deb
```

If you do not have aria2c:

```
wget https://github.com/NVIDIA/DeepStream/releases/download/v9.1.0/deepstream-9.1_9.1.0-1_arm64.deb
```

Verify the download:

```
ls -lh /tmp/deepstream-9.1_9.1.0-1_arm64.deb
```

---

3. Install the TensorRT 10.x dependencies

DeepStream 9.1 requires TensorRT 10.x, while JetPack 7.2.1 defaults to 11.x

```
sudo apt update
sudo apt install -y \
  libnvinfer10=10.16.2.10-1+cuda13.2 \
  libnvonnxparsers10=10.16.2.10-1+cuda13.2 \
  libnvinfer-plugin10=10.16.2.10-1+cuda13.2
```

Verify the installation:

```
dpkg -l | grep libnvinfer10
# It should show: ii  libnvinfer10  10.16.2.10-1+cuda13.2
```

---

4. Install DeepStream 9.1

```
sudo dpkg -i /tmp/deepstream-9.1_9.1.0-1_arm64.deb
sudo apt install -f -y
```

Verify the installation:

```
dpkg -l | grep deepstream
# It should show: ii  deepstream-9.1  9.1.0-1  arm64
```

Configure DeepStream:

```
sudo dpkg --configure deepstream-9.1
```

---

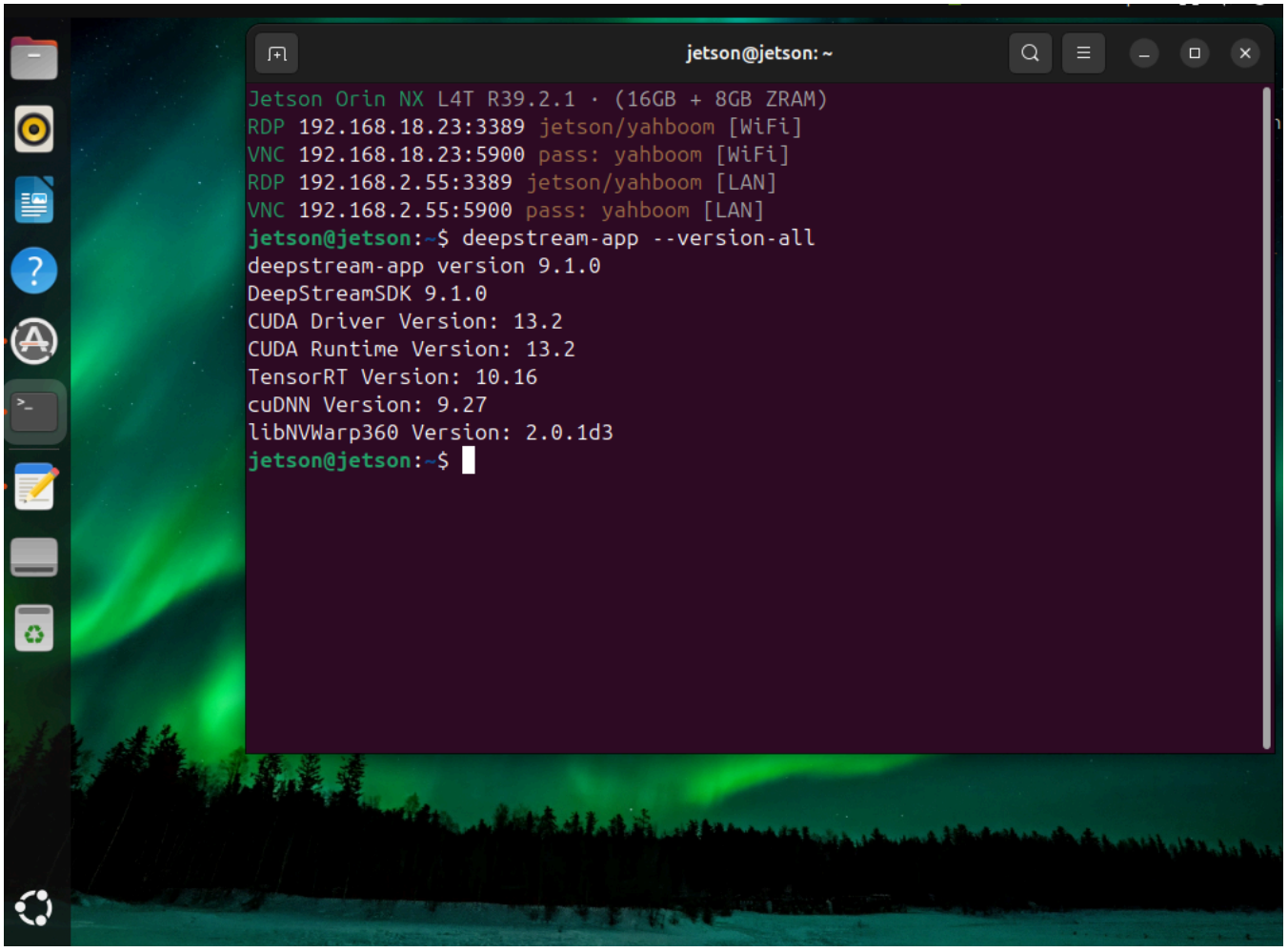
5. Verify the installation

Check the versions of DeepStream, CUDA and TensorRT:

```
deepstream-app --version-all
```

It is expected to contain:

```
deepstream-app version 9.1.0
DeepStreamSDK 9.1.0
CUDA Driver Version: 13.2
CUDA Runtime Version: 13.2
TensorRT Version: 10.16
cuDNN Version: 9.27
libNVWarp360 Version: 2.0.1d3
```



In general, if cuDNN is shown as unavailable, that is a display bug and it actually works; if you want the cuDNN version to be displayed as in the image, you need

```
sudo apt install libcudnn9-dev-cuda-13
```

Confirm the status of the packages and of the core GStreamer plugins:

```
dpkg-query -W -f='${Status} ${Version}\n' deepstream-9.1

gst-inspect-1.0 nvinfer >/dev/null && echo "nvinfer OK"
gst-inspect-1.0 nvstreammux >/dev/null && echo "nvstreammux OK"
gst-inspect-1.0 nvv4l2decoder >/dev/null && echo "hardware decoder plugin OK"
```

6. Run the basic video detection case

This case uses the traffic video and TrafficCamNet that come with DeepStream; the terminal continuously outputs the number of vehicles and people in each frame.

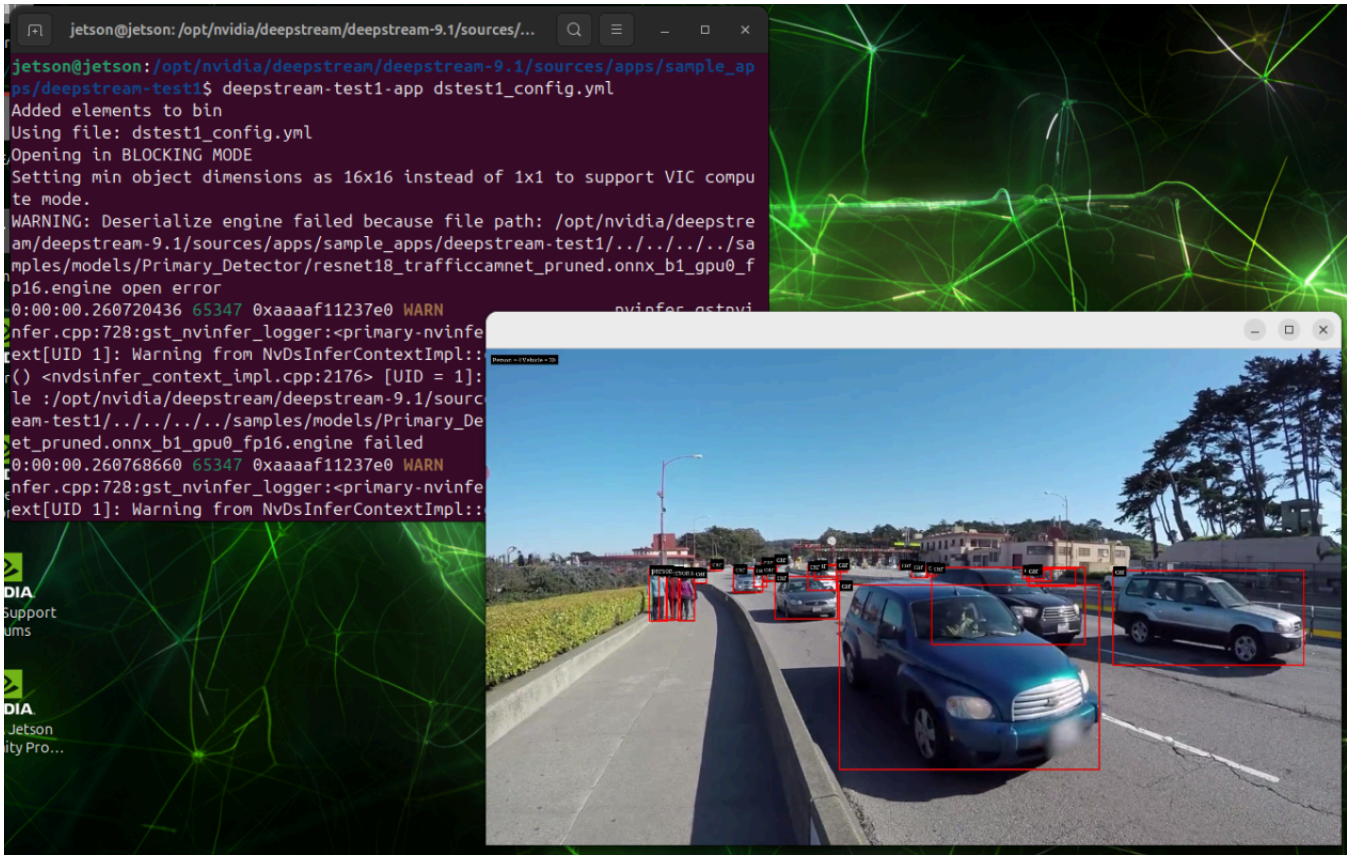
```
cd /opt/nvidia/deepstream/deepstream-9.1/sources/apps/sample_apps/deepstream-test1
deepstream-test1-app dstest1_config.yml
```

The full path of the test video is:

```
/opt/nvidia/deepstream/deepstream-9.1/samples/streams/sample_720p.h264
```

Signs of success:

```
Frame Number = 0 Number of objects = ... Vehicle Count = ... Person Count = ...
...
End of stream
Returned, stopping playback
Deleting pipeline
```



The following message appearing on the first run is normal:

```
Deserialize engine failed
Trying to create engine from model files
```

This means that the corresponding TensorRT engine does not exist on the device yet, and DeepStream is building the engine from the ONNX model. The first build takes a while.

If `failed to serialize cuda engine to file` appears, it is usually because the normal user does not have the write permission for `/opt/nvidia/deepstream/.../samples/models`; this inference can still run normally, only the later startups may rebuild the engine.

7. USB and CSI camera detection

Run it in a graphical desktop, a VNC or an RDP session. A pure SSH session has no display environment, so the `nv3d-sink` graphical output cannot be verified.

7.1. USB camera

You can confirm the camera nodes first:

```
ls -l /dev/video*
```

Enter the following commands to preview

```
cd ~/deepstream_tao_demo
./run_camera_preview.sh usb
./run_case.sh peoplenet usb
```

The preview script selects the resolution and the frame rate according to the formats actually supported by the camera. If `not-negotiated` appears, do not manually modify several templates; using this entry point avoids an inconsistency of the node, the resolution or the frame rate.

7.2. CSI camera

```
cd ~/deepstream_tao_demo
./run_camera_preview.sh csi
./run_case.sh peoplenet csi
```

When the CSI works, the terminal lists the sensor modes supported by Argus and continuously outputs the FPS. If Argus can recognize the sensor but the pop-up window is black and then exits, check the CSI camera module, the ribbon cable and the interface first; in the test of this time it ran normally after the CSI camera was replaced.

The CSI preview requests `sensor-mode=3`, `wbmode=1` and an exposure compensation of 0 by default. If the current
`nvarguscamerasrc` has no `sensor-mode` property, then the `1920x1080@30` caps
is used to select the sensor mode. `wbmode=3` is the white balance of the fluorescent lamp and not the exposure mode, and under the normal lighting it may cause a red or a magenta cast.

8. Clean up the installation package

After the installation and the verification are complete, you can delete the temporary installation package and the download log in `/tmp` so that they do not take up space:

```
rm -f /tmp/deepstream-9.1_9.1.0-1_arm64.deb \
  /tmp/deepstream-9.1_9.1.0-1_arm64.deb.aria2 \
  /tmp/deepstream-9.1-download.log
```

Do not run `sudo apt autoremove` unless you have already confirmed that the automatically installed packages that are listed are no longer used by other courses or projects.

9. Common problems

Problem 1: aria2c cannot be found

Solution:

```
sudo apt install -y aria2
```

Problem 2: dependency error

Solution:

```
sudo apt install -f -y
```

Problem 3: TensorRT version conflict

Make sure the correct version is installed:

```
dpkg -l | grep libnvinfer
# It should show libnvinfer10, not libnvinfer11
```
